



Bayesian Optimization

Chapter Summary (Sections 1–4)

Bernd Bischl





Overview: Chapter 4 – Bayesian Optimization (Part 1)

This summary covers the first four sections:

1. **The Big Picture** – Why BO? The black-box HPO problem
2. **The Basic BO Loop** – Surrogate modeling, initial design, fit-propose-evaluate
3. **Gaussian Process Surrogates** – Model, posterior, kernels, learning
4. **Random Forest Surrogates** – Uncertainty estimation for tree ensembles

Central challenge: Find λ^* of an *expensive, noisy, black-box* objective with as few evaluations as possible.



Why Bayesian Optimization?

HPO is an expensive black-box problem:

$$\lambda^* \in \arg \min_{\lambda \in \Lambda} c(\lambda), \quad c(\lambda) = \mathcal{R}_{\text{gen}}(\mathcal{I}(\mathcal{D}_{\text{train}}, \lambda), \mathcal{D}_{\text{val}})$$

- **Black box** – no analytic form, no gradients available
- **Expensive** – each evaluation trains / resamples a model (minutes to days)
- **Noisy** – stochastic training, resampling, and measurement noise
- **Mixed / hierarchical Λ** – numeric, integer, categorical, and conditional HPs

Why not simpler alternatives?

Manual tuning	Biased, not reproducible, not automated
Grid search	Exponential scaling; evaluates irrelevant areas
Random search	Better coverage, but still non-adaptive
Bayesian optimization	Adaptive, sample-efficient, principled



The Core Idea of BO

1. Fit a cheap **surrogate model** $\hat{c}$ to the observations collected so far
2. Optimize a cheap **acquisition function** $u(\lambda)$ that trades off
 - **Exploitation**: high where $\hat{c}$ predicts good values
 - **Exploration**: high where $\hat{c}$ is uncertain
3. Evaluate the expensive objective *only* at the proposed point; update $\hat{c}$; repeat

Why it works: each evaluation is guided by everything learned so far – budget is concentrated in promising, uncertain regions.

Applications beyond HPO: robotics controller tuning, material / drug design, simulator calibration, experimental cooking recipes – any costly, gradient-free problem.



The Basic BO Loop

Step	Action	Details
0	Initial design	Space-filling batch; covers Λ before first surrogate fit
1	Fit $\hat{c}$	Regression on archive $\mathcal{D}_{\text{Hist}} = \{(\boldsymbol{\lambda}^{(i)}, c(\boldsymbol{\lambda}^{(i)}))\}_{i=1}^T$
2	Propose $\boldsymbol{\lambda}^{(T+1)}$	Maximize acquisition function $u(\boldsymbol{\lambda})$ over Λ
3	Evaluate $c(\boldsymbol{\lambda}^{(T+1)})$	One expensive model training / resampling
4	Update $\mathcal{D}_{\text{Hist}}$	Append new observation; go to step 1

Core design choices: initial design strategy, surrogate model, acquisition function.

Pure exploitation (propose next = $\arg \min_{\boldsymbol{\lambda}} \hat{c}(\boldsymbol{\lambda})$) has no exploration and risks getting trapped in a local minimum – acquisition functions are needed.



Initial Design

The first batch of evaluations must **cover** Λ before the surrogate can be trusted.

Latin Hypercube Sampling (LHS):

- Partition each dimension into n equal bins; place exactly 1 point per row & column
- Much better marginal coverage than random sampling
- Quality can be improved by a **maximin** criterion: choose the LHS that maximises

$$\max_{\text{design}} \min_{\lambda \neq \lambda' \in \text{design}} \rho(\lambda, \lambda')$$

i.e. spread points as far apart as possible (maximize the packing radius)

Practical rule of thumb: initial design size $\approx 4d$, where $d = \dim(\Lambda)$.

- Too small: misleading initial surrogate; too large: budget wasted before adaptive search starts



Gaussian Process Surrogates

A GP is a distribution over functions. For any finite set of inputs, the joint distribution of function values is Gaussian – fully specified by mean $m(\mathbf{x})$ and kernel $k(\mathbf{x}, \mathbf{x}')$.

Observation model: $y^{(i)} = f(\mathbf{x}^{(i)}) + \epsilon_i$, $\epsilon_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma_\epsilon^2)$.

Posterior predictive at a new point $\mathbf{x}$ (conditioning on observed data $\mathcal{D}$):

$$Y(\mathbf{x}) \mid \mathbf{x}, \mathcal{D} \sim \mathcal{N}(\hat{\mu}(\mathbf{x}), \hat{\sigma}^2(\mathbf{x}))$$

$$\hat{\mu}(\mathbf{x}) = \mathbf{k}_*^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{y}, \quad \hat{\sigma}^2(\mathbf{x}) = k_{**} - \mathbf{k}_*^T (\mathbf{K} + \sigma_\epsilon^2 \mathbf{I})^{-1} \mathbf{k}_*$$

- $\hat{\mu}(\mathbf{x})$: exploits training labels via kernel-weighted linear combination
- $\hat{\sigma}^2(\mathbf{x})$: prior variance minus what data resolved – **GP “knows what it doesn’t know”**
- With $\sigma_\epsilon^2 = 0$: exact interpolation; $\hat{\sigma}^2(\mathbf{x}^{(i)}) = 0$ at training points



Kernels and Hyperparameter Learning

The kernel $k(\mathbf{x}, \mathbf{x}')$ encodes assumptions about smoothness and length scale.

Kernel	Differentiability	Remarks
RBF / Squared Exponential	∞	Too smooth for realistic response surfaces
Matérn-1/2	0	Brownian-like, rough
Matérn-3/2	1	Moderately wiggly
Matérn-5/2	2	Default for BO

ARD: Per-dimension length scales l_j are inferred from data;
small $l_j \Rightarrow$ dimension j is relevant.

Learning kernel HPs θ (length scales, noise σ_ϵ^2): maximize the *log marginal likelihood*

$$\log p(\mathbf{y} \mid \theta) = -\frac{1}{2} \mathbf{y}^T \mathbf{K}_\theta^{-1} \mathbf{y} - \frac{1}{2} \log |\mathbf{K}_\theta| - \frac{N}{2} \log 2\pi$$

via L-BFGS-B (multi-start). Training cost: $\mathcal{O}(N^3)$ per evaluation.



GPs: Mixed Spaces and Pros / Cons

Vanilla GPs require numeric inputs – workarounds for AutoML:

- **Integer HPs:** treat as continuous; round to nearest integer before evaluation
- **Categorical HPs:** one-hot encoding *or* Hamming/overlap kernel
 $k_{\text{cat}}(c, c') = \mathbb{1}[c = c']$, combined with the numeric kernel as a product
- **Conditional / hierarchical HPs:** impute inactive HPs with sentinel values; dedicated arc / tree-structured kernels exist but are rarely used in practice

	Pros	Cons
GP	Well-calibrated uncertainty; strong small-N performance; closed-form posteriors	$\mathcal{O}(N^3)$ training; needs numeric inputs natively; sensitive to kernel choice



Random Forest Surrogates

Motivation: GPs struggle with mixed/hierarchical Λ and scale poorly with N .

Random Forests handle these naturally:

- Tree splits work directly on categorical and conditional features (no kernel needed)
- Training: $\mathcal{O}(B \cdot N \log N)$; robust, low hyperparameter sensitivity
- Used by **SMAC** (Hutter et al. 2011) – the backbone of Auto-WEKA and auto-sklearn

Uncertainty challenge: RFs are not Bayesian – variance must be estimated externally.

Estimator	Key idea
Naive between-tree	Variance of B per-tree predictions
Law of total variance	Within-leaf (aleatoric) + between-tree (epistemic)
Infinitesimal jackknife	Covariance of bootstrap counts with predictions
Quantile RF	Full empirical leaf distribution; arbitrary quantiles



RF Variance in Detail

Law of total variance (Hutter et al. 2014; used in SMAC):

Let $\hat{f}_b(\mathbf{x})$ = prediction of tree b ; $\hat{\sigma}_b^2(\mathbf{x})$ = within-leaf variance of tree b :

$$\hat{\sigma}^2(\mathbf{x}) = \underbrace{\frac{1}{B} \sum_{b=1}^B \hat{\sigma}_b^2(\mathbf{x})}_{\text{within-tree (aleatoric)}} + \underbrace{\frac{1}{B} \sum_{b=1}^B \hat{f}_b(\mathbf{x})^2 - \hat{f}(\mathbf{x})^2}_{\text{between-tree (epistemic)}}$$

Extra-trees (Geurts et al. 2006): choose split points uniformly at random per candidate feature instead of searching for the empirically best cut.

- Faster training; more between-tree disagreement $\rightsquigarrow$ smoother mean $\hat{f}$

Calibration note: RF variance tends to **underestimate** uncertainty in extrapolation zones and **overestimate** near training data.



GP vs. RF Surrogate: When to Use Which

Property	GP	RF
Numeric-only Λ	Preferred	Works
Mixed / conditional Λ	Needs special kernels	Preferred
Small N ($\lesssim 200$)	Preferred	Works
Large N	Scales poorly ($\mathcal{O}(N^3)$)	Preferred
Calibrated uncertainty	Yes (Bayesian)	Approximate
Gradient-based AF opt.	Yes	Piecewise-constant
Robustness / ease of use	Moderate	High

Practical rule: GPs for low-dimensional numeric HPO with tight budget; RFs for CASH / AutoML pipelines or larger budgets with categorical / conditional hyperparameters.



Chapter Summary (Sections 1-4)

Key takeaways:

1. HPO is an expensive, noisy, black-box problem; manual tuning and non-adaptive search waste budget on uninformative evaluations
2. BO concentrates evaluations in promising, uncertain regions by iterating: **fit surrogate** → **optimize acquisition function** → **evaluate** → **update**
3. A good initial design (LHS + maximin, size $\approx 4d$) is essential for a reliable first surrogate; optimizing the surrogate mean alone leads to pure exploitation
4. GPs provide closed-form, well-calibrated posteriors and are the default surrogate for numeric, low-dimensional spaces; kernel (Matérn-5/2 + ARD) and its hyperparameters are learned via marginal likelihood
5. Random Forests handle mixed / hierarchical configuration spaces naturally; uncertainty is extracted via between-tree disagreement, law of total variance, or quantile forests; used in SMAC and related AutoML systems